CortextGuard Workflow (High-Level Architecture)

1. Data Ingestion Layer (Cloud + DevOps Sources)

CortextGuard continuously collects data from multiple sources:

- Cloud providers: AWS, Azure, GCP
- IaC tools: Terraform, CloudFormation
- CI/CD pipelines: GitHub Actions, Jenkins
- Containers: Docker, Kubernetes
- Logs & configs: IAM policies, network rules, storage permissions

How it works:

- API-based connectors + webhook listeners
- Scheduled scans + real-time event streaming

---

2. Normalization & Context Building

Raw data is messy — this layer standardizes everything.

- Convert configs into a **common schema**
- Build a **resource relationship graph**
    - Example: EC2 → Security Group → Open Port → Internet exposure
- Tag assets with:
    - Environment (prod/dev)
    - Ownership
    - Sensitivity level

Output: **Context-aware cloud inventory**

---

3. Misconfiguration Detection Engine

This is your core security scanning layer.

A. Rule-Based Detection

- CIS Benchmarks
- OWASP best practices
- Custom org policies

B. AI-Based Detection (Agentic Layer)

- LLM agents analyze configs semantically
- Detect:
    - Over-permissive IAM roles
    - Publicly exposed resources
    - Privilege escalation paths

Example:

> "S3 bucket is public AND linked to sensitive data → high risk"

---

4. Agentic AI Decision Layer

This is what makes CortextGuard *agentic*.

Multiple AI agents collaborate:

Agents:

- **Analyzer Agent** → Understands the issue
- **Risk Agent** → Assigns severity (context-aware)
- **Fix Generator Agent** → Suggests remediation
- **Policy Agent** → Ensures compliance alignment

Behavior:

- Multi-step reasoning (chain-of-thought internally)
- Tool usage (query logs, configs, past incidents)
- Feedback loops

Output:

- Root cause
- Risk score
- Actionable fix

---

5. Prioritization & Risk Scoring

Not all issues matter equally.

CortextGuard prioritizes using:

- Exposure level (public/private)
- Data sensitivity
- Attack path likelihood
- Business impact

Produces:

- **Critical alerts first**
- Attack path visualization

---

6. Automated Remediation Engine

Two modes:

A. Suggestive Fixes

- Terraform patch suggestions
- CLI commands
- YAML updates

B. Auto-Remediation (optional)

- Close open ports
- Restrict IAM roles
- Disable public access

Safety:

- Approval workflows
- Rollback support

---

7. DevSecOps Integration

Embed security into development lifecycle:

- PR scanning (before merge)
- CI/CD pipeline checks
- Policy-as-code enforcement

Example:

- Block deployment if misconfiguration detected

---

8. Dashboard & Visualization (Frontend)

Full-stack UI provides:

- Asset inventory
- Active vulnerabilities
- Risk trends
- Attack path graphs

Tech stack example:

- Frontend: React / Next.js
- Backend: Node.js / FastAPI
- Graph DB: Neo4j (for relationships)

---

9. Alerting & Reporting

- Slack / Email / Teams alerts
- Compliance reports (SOC2, ISO 27001)
- Executive summaries (AI-generated)

---

10. Continuous Learning Loop

System improves over time:

- Learns from:
    - Past incidents
    - User feedback
    - Fix success rates
- Updates:
    - Detection rules
    - Risk models
    - AI prompts

End-to-End Flow Summary

```
Cloud & DevOps Data
        ↓
```

```
Normalization & Context Graph
        ↓
Misconfiguration Detection
        ↓
Agentic AI Analysis
        ↓
Risk Prioritization
        ↓
Remediation (Suggest / Auto-Fix)
        ↓
DevSecOps Integration + Alerts
        ↓
Dashboard + Continuous Learning
```

Key Differentiators of CortextGuard

- Agentic AI (not just static scanning)
- Context-aware risk scoring
- Automated + explainable remediation
- Graph-based attack path analysis
- Full DevSecOps integration

If you want, I can also:

- Turn this into a **system design diagram**
- Provide **tech stack with tools & APIs**
- Help you write a **project report / PPT**
- Or give **code architecture (microservices breakdown)**

provide me workflow in easy steps also with weekwise work distribution

Got it — let's simplify your **CortextGuard** workflow into *easy, practical steps* and then break it into a **week-by-week execution plan** you can actually follow.

✅ Simple Workflow (Step-by-Step)

Think of CortextGuard as a pipeline with 7 simple steps:

1. Collect Data

- Connect to cloud (AWS / Azure / GCP)
- Pull configs (IAM, storage, network)
- Fetch CI/CD + IaC files

Output: Raw cloud + DevOps data

---

2. Clean & Organize Data

- Convert all configs into one format
- Identify resources (VMs, buckets, roles)
- Add context (public/private, env, owner)

Output: Structured cloud inventory

---

3. Detect Security Issues

- Apply rules:
    - Open ports
    - Public storage
    - Weak IAM roles

Output: List of misconfigurations

---

4. AI Analysis (Agent Layer)

- AI understands each issue
- Finds:
    - Why it's risky
    - How attackers could exploit it

Output: Smart explanation + root cause

---

5. Risk Prioritization

- Rank issues:
    - Critical
    - Medium
    - Low

Based on:

- Exposure + sensitivity

---

6. Fix Suggestions / Auto-Fix

- Suggest:
    - Terraform fix
    - CLI commands
- Optional:
    - Auto-remediate issues

---

7. Dashboard & Alerts

- Show:
    - Issues
    - Risk score
    - Fix suggestions
- Send alerts (Slack/Email)

One-Line Flow

Collect → Clean → Detect → AI Analyze → Prioritize → Fix → Display

---

Week-wise Work Plan (8 Weeks)

This is structured like a real academic / hackathon / startup build timeline.

Week 1: Project Setup & Research

Goal: Understand problem + set foundation

- Finalize features of CortextGuard
- Research:
  - Cloud misconfigurations (CIS benchmarks)
  - DevSecOps tools
- Choose tech stack:
  - Frontend: React / Next.js
  - Backend: Node.js / FastAPI
- Setup GitHub repo + project structure

Output: Clear roadmap + environment ready

Week 2: Data Collection Module

Goal: Fetch cloud/config data

- Build API connectors (start with AWS mock or sample JSON)
- Parse:
  - IAM roles
  - S3 buckets
  - Security groups

Output: Raw cloud data ingestion working

Week 3: Data Processing & Context Layer

Goal: Make data usable

- Normalize data format
- Create resource mapping:
  - Resource → permissions → exposure
- Tag:
  - Public/private
  - Sensitive resources

Output: Structured dataset

Week 4: Detection Engine

Goal: Find misconfigurations

- Implement rule-based checks:
  - Public S3 bucket
  - Open ports (0.0.0.0/0)
  - Admin IAM roles

Output: Issue detection system ready

Week 5: AI Agent Integration

Goal: Add intelligence

- Integrate LLM (OpenAI or local model)
- Build agents:
  - Analyzer → explain issue
  - Fix generator → suggest solution

Output: AI explanations + fixes

Week 6: Risk Scoring & Prioritization

Goal: Rank issues smartly

- Define scoring logic:
  - Exposure + sensitivity + impact
- Categorize:
  - Critical / Medium / Low

Output: Prioritized vulnerabilities

Week 7: Frontend Dashboard

Goal: Visual interface

- Build UI:
    - Issue list
    - Risk levels
    - Fix suggestions
- Add charts (risk trends)

Output: Working dashboard

Week 8: Integration + Testing + Final Touch

Goal: Make it production-ready

- Connect all modules
- Add:
    - Alerts (email/Slack)
    - Auto-fix (optional)
- Test with real scenarios
- Prepare demo + presentation

Output: Complete working project